# TOPIC 2 : BRUTE FORCE

## 21. Given an array arr of positive integers sorted in strictly increasing order, and an integer k, return the kth positive integer that is missing from this array. Example 1: arr = [2,3,4,7,11], k = 5 -> Output: 9 Example 2: arr = [1,2,3,4], k = 2 -> Output: 6

### Aim

To write a Python program to find the k-th missing positive integer from a sorted array.

### Algorithm

1. Start.
2. Iterate through arr.
3. If num <= k, increment k by 1.
4. Else, break loop.
5. Return k.
6. Stop.

main.py Run

```python
def findKthPositive(arr, k):
    for num in arr:
        if num <= k:
            k += 1
        else:
            break
    return k

print(findKthPositive([2, 3, 4, 7, 11], 5))
print(findKthPositive([1, 2, 3, 4], 2))
```

Output

```
9
6
```

### Time Complexity

O(n) linear scan or O(log n) binary search.

### Space Complexity

O(1) auxiliary space.

## 22. A peak element is strictly greater than its neighbors. Given array nums, find a peak element and return its index. Example 1: nums = [1,2,3,1] -> Output: 2 (element 3) Example 2: nums = [1,2,1,3,5,6,4] -> Output: 5 (element 6)

### Aim

To write a Python program to locate any peak element in an array in O(log n) time.

### Algorithm

1. Start.
2. Initialize low = 0, high = len(nums) - 1.
3. While low < high:
4. mid = (low + high) // 2
5. If nums[mid] > nums[mid + 1]: high = mid
6. Else: low = mid + 1
7. Return low.
8. Stop.

main.py Run

```python
def findPeakElement(nums):
    low, high = 0, len(nums) - 1
    while low < high:
        mid = (low + high) // 2
        if nums[mid] > nums[mid + 1]:
            high = mid
        else:
            low = mid + 1
    return low

print(findPeakElement([1, 2, 3, 1]))
print(findPeakElement([1, 2, 1, 3, 5, 6, 4]))
```

Output

```
2
5
```

### Time Complexity

O(log n) binary search.

### Space Complexity

O(1) space.

**23. Given haystack and needle strings, return index of first occurrence of needle in haystack, or -1 if needle is not part of haystack. Example 1: haystack = "sadbutsad", needle = "sad" -> Output: 0 Example 2: haystack = "leetcode", needle = "leeto" -> Output: -1**

### Aim

To write a Python program using brute force string matching to find first occurrence of needle substring.

### Algorithm

1. Start.
2. Loop i from 0 to len(haystack) - len(needle) + 1.
3. If haystack[i : i+len(needle)] == needle, return i.
4. If loop finishes without match, return -1.
5. Stop.

main.py Run

```python
def strStr(haystack, needle):
    n, m = len(haystack), len(needle)
    for i in range(n - m + 1):
        if haystack[i : i + m] == needle:
            return i
    return -1

print(strStr("sadbutsad", "sad"))
print(strStr("leetcode", "leeto"))
```

Output

```
0
-1
```

### Time Complexity

O((n - m) × m) brute force substring matching.

### Space Complexity

O(1) constant auxiliary space.

## 24. Given array words, return all strings in words that are substrings of another word in any order. Example 1: words = ["mass","as","hero","superhero"] -> Output: ["as","hero"] Example 2: words = ["leetcode","et","code"] -> Output: ["et","code"]

### Aim

To write a Python program to find all words that are substrings of other words in array.

### Algorithm

1. Start.
2. Initialize empty list res.
3. Outer loop word1 in words.
4. Inner loop word2 in words.
5. If word1 != word2 and word1 in word2, append word1 to res and break.
6. Return res.
7. Stop.

main.py Run

```python
def stringMatching(words):
    res = []
    for i, w1 in enumerate(words):
        for j, w2 in enumerate(words):
            if i != j and w1 in w2:
                res.append(w1)
                break
    return res

print(stringMatching(["mass", "as", "hero", "superhero"]))
print(stringMatching(["leetcode", "et", "code"]))
```

Output

```
['as', 'hero']
['et', 'code']
```

### Time Complexity

$O(n^2 \times m^2)$ where n is number of words and m is max word length.

### Space Complexity

O(n) auxiliary space for output.

## 25. Write a program that finds the closest pair of points in a set of 2D points using the brute force approach. Input: Points = [(1, 2), (4, 5), (7, 8), (3, 1)] Output: Closest pair: (1, 2) - (3, 1), Minimum distance: 1.4142135623730951

### Aim

To write a Python program to compute minimum Euclidean distance between all pairs of 2D points.

### Algorithm

1. Start.
2. Initialize min_dist = float('inf'), pair = None.
3. Loop i from 0 to n-1, j from i+1 to n-1.
4. Compute distance d = sqrt((x2 - x1)² + (y2 - y1)²).
5. If d < min_dist, update min_dist = d and pair = (p1, p2).
6. Return pair and min_dist.
7. Stop.

main.py Run

```python
import math

def closest_pair(points):
    min_dist = float('inf')
    closest = None
    n = len(points)
    for i in range(n):
        for j in range(i + 1, n):
            d = math.dist(points[i], points[j])
            if d < min_dist:
                min_dist = d
                closest = (points[i], points[j])
    return closest, min_dist

pts = [(1, 2), (4, 5), (7, 8), (3, 1)]
pair, dist = closest_pair(pts)
print(f"Closest pair: {pair[0]} - {pair[1]} Minimum distance: {dist}")
```

Output

```
Closest pair: (1, 2) - (3, 1) Minimum distance: 2.23606797749979
```

### Time Complexity

O(n²) exhaustive pairwise comparisons.

### Space Complexity

O(1) constant auxiliary space.

## 26. Write a program to find closest pair of points for given points: P1(10,0), P2(11,5), P3(5,3), P4(9,3.5), P5(15,3), P6(12.5,7), P7(6,6.5), P8(7.5,4.5). Output: P3, P4, P6, P5, P7, P1

### Aim

To write a Python program to analyze closest point pairs using Euclidean distance formulas.

### Algorithm

1. Start.
2. Define Euclidean distance function.
3. Compute all pairwise distances for point set S.
4. Track point pairs yielding minimum geometric distance.
5. Return minimum distance pair.
6. Stop.

main.py Run

```python
import math

points = [(10, 0), (11, 5), (5, 3), (9, 3.5), (15, 3), (12.5, 7), (6, 6.5), (7.5, 4.5)]

min_d = float('inf')
best_pair = None
for i in range(len(points)):
    for j in range(i + 1, len(points)):
        d = math.dist(points[i], points[j])
        if d < min_d:
            min_d = d
            best_pair = (f"P{i+1}", f"P{j+1}")

print(f"Closest points: {best_pair[0]} and {best_pair[1]} with distance: {min_d:.4f}")
```

Output

```
Closest points: P4 and P8 with distance: 1.8028
```

### Time Complexity

$O(n^2)$ time complexity.

### Space Complexity

O(1) space complexity.

## 27. Write a program that finds the convex hull of a set of 2D points using brute force approach. Input: Points = [(1, 1), (4, 6), (8, 1), (0, 0), (3, 3)] Output: Convex Hull: [(0, 0), (1, 1), (8, 1), (4, 6)]

### Aim

To write a Python program to find boundary points forming Convex Hull of a 2D point cloud.

### Algorithm

1. Start.
2. For every pair of points (p1, p2):
3. Check if all other points lie on the same side of line p1-p2.
4. Line equation: (y2 - y1)*(x - x1) - (x2 - x1)*(y - y1).
5. If all other points have same sign, edge (p1, p2) belongs to Convex Hull.
6. Return sorted hull vertices in counter-clockwise order.
7. Stop.

main.py Run

```python
def brute_force_convex_hull(points):
    n = len(points)
    hull_edges = set()
    for i in range(n):
        for j in range(n):
            if i == j: continue
            p1, p2 = points[i], points[j]
            valid = True
            pos, neg = 0, 0
            for k in range(n):
                if k == i or k == j: continue
                p3 = points[k]
                val = (p2[1] - p1[1]) * (p3[0] - p1[0]) - (p2[0] - p1[0]) * (p3[1] - p1[1])
                if val > 0: pos += 1
                elif val < 0: neg += 1
                if pos > 0 and neg > 0:
                    valid = False
                    break
            if valid:
                hull_edges.add(p1)
                hull_edges.add(p2)
    return sorted(list(hull_edges))

pts = [(1, 1), (4, 6), (8, 1), (0, 0), (3, 3)]
print("Convex Hull:", brute_force_convex_hull(pts))
```

Output

```
Convex Hull: [(0, 0), (4, 6), (8, 1)]
```

### Time Complexity

$O(n^3)$ brute force triple loop checking all points against line segments.

### Space Complexity

O(n) hull set storage.

## 28. Develop a program that utilizes exhaustive search to solve TSP. Test Case 1: Cities [(1, 2), (4, 5), (7, 1), (3, 6)] -> Output Shortest Distance: 17.65, Path: [(1, 2), (7, 1), (4, 5), (3, 6), (1, 2)] Test Case 2: Cities [(2, 4), (1, 7), (6, 3), (5, 9), (8, 1)] -> Output Shortest Distance: 25.14

### Aim

To write a Python program using itertools.permutations to solve TSP by evaluating all possible tour permutations.

### Algorithm

1. Start.
2. Generate all permutations of intermediate cities.
3. Fix starting city as first city.
4. For each permutation, calculate total tour distance back to start.
5. Keep track of permutation yielding minimum tour length.
6. Return shortest path and total distance.
7. Stop.

main.py Run

```python
import itertools, math

def solve_tsp(cities):
    n = len(cities)
    start_city = cities[0]
    rest_cities = cities[1:]

    min_dist = float('inf')
    best_path = None

    for perm in itertools.permutations(rest_cities):
        path = [start_city] + list(perm) + [start_city]
        dist = sum(math.dist(path[k], path[k+1]) for k in range(n))
        if dist < min_dist:
            min_dist = dist
            best_path = path

    return min_dist, best_path

c1 = [(1, 2), (4, 5), (7, 1), (3, 6)]
d1, p1 = solve_tsp(c1)
print(f"Shortest Distance: {d1:.2f}")

c2 = [(2, 4), (1, 7), (6, 3), (5, 9), (8, 1)]
d2, p2 = solve_tsp(c2)
print(f"Shortest Distance: {d2:.2f}")
```

Output

```
Shortest Distance: 16.97
Shortest Distance: 23.13
```

### Time Complexity

O((n-1)!) factorial exhaustive permutation search.

### Space Complexity

O(n) path storage space.

## 29. Given a cost matrix where each element cost[i][j] represents cost of assigning worker i to task j. Solve Assignment Problem using brute force permutations. Simple Case: Cost Matrix [[3,10,7], [8,5,12], [4,6,9]] -> Optimal Cost: 19 Complex Case: Cost Matrix [[15,9,4], [8,7,18], [6,12,11]] -> Optimal Cost: 24

### Aim

To write a Python program to find minimum cost assignment of workers to tasks using exhaustive permutation evaluation.

### Algorithm

1. Start.
2. Read n x n cost matrix.
3. Generate all permutations of task assignments range(n).
4. Calculate total cost sum(cost[i][perm[i]]) for each permutation.
5. Identify assignment permutation yielding minimal cost.
6. Return optimal assignments and total minimum cost.
7. Stop.

main.py Run

```python
import itertools

def assignment_problem(cost_matrix):
    n = len(cost_matrix)
    min_cost = float('inf')
    best_assignment = None

    for perm in itertools.permutations(range(n)):
        current_cost = sum(cost_matrix[i][perm[i]] for i in range(n))
        if current_cost < min_cost:
            min_cost = current_cost
            best_assignment = perm

    return best_assignment, min_cost

cost1 = [[3, 10, 7], [8, 5, 12], [4, 6, 9]]
assign1, total1 = assignment_problem(cost1)
print(f"Total Cost: {total1}")

cost2 = [[15, 9, 4], [8, 7, 18], [6, 12, 11]]
assign2, total2 = assignment_problem(cost2)
print(f"Total Cost: {total2}")
```

Output

```
Total Cost: 16
Total Cost: 17
```

### Time Complexity

O(n!) factorial search across task permutations.

### Space Complexity

O(n) auxiliary permutation memory.

## 30. Develop a program that utilizes exhaustive search to solve 0-1 Knapsack Problem. Simple Case: Weights [2,3,1], Values [4,5,3], Capacity = 4 -> Output Optimal Selection: [0, 2], Total Value: 7 Complex Case: Weights [1,2,3,4], Values [2,4,6,3], Capacity = 6 -> Output Optimal Selection: [0, 1, 2], Total Value: 10

### Aim

To write a Python program to evaluate all 2■ subsets of items to solve 0-1 Knapsack Problem.

### Algorithm

1. Start.
2. Read item weights, values, and capacity W.
3. Loop subset selection mask from 0 to 2■ - 1.
4. For each subset, calculate total weight and total value.
5. If total weight <= W and total value > max_val, update max_val and optimal selection.
6. Return best subset indices and maximum total value.
7. Stop.

main.py Run

```python
def knapsack_brute_force(weights, values, capacity):
    n = len(weights)
    max_value = 0
    best_comb = []

    for i in range(1 << n):
        curr_weight = 0
        curr_value = 0
        comb = []
        for j in range(n):
            if (i >> j) & 1:
                curr_weight += weights[j]
                curr_value += values[j]
                comb.append(j)
        if curr_weight <= capacity and curr_value > max_value:
            max_value = curr_value
            best_comb = comb

    return best_comb, max_value

w1, v1, cap1 = [2, 3, 1], [4, 5, 3], 4
c1, val1 = knapsack_brute_force(w1, v1, cap1)
print(f"Optimal Selection: {c1}, Total Value: {val1}")

w2, v2, cap2 = [1, 2, 3, 4], [2, 4, 6, 3], 6
c2, val2 = knapsack_brute_force(w2, v2, cap2)
print(f"Optimal Selection: {c2}, Total Value: {val2}")
```

Output

```
Optimal Selection: [1, 2], Total Value: 8
Optimal Selection: [0, 1, 2], Total Value: 12
```

### Time Complexity

O(2■ × n) exponential brute force power set evaluation.

### Space Complexity

O(n) subset array storage.